

CitGraph: Citation Intent Classification and Second-Hand Citation Detection via Knowledge Graphs

Giorgio Scavello

Lorenzo Pellegrino

giorgio.scavello@studio.unibo.it

lorenzo.pellegrino2@studio.unibo.it

0001169261

0001189550

Abstract

A citation does not exist in isolation: its meaning is shaped by the network of papers that surround it. A paper that is consistently cited for its methodology looks different in the citation graph from one that is consistently cited as background or as a result to compare against and a citation that does not fit its local graph neighbourhood is itself a signal worth investigating.

1 Description

This project builds CitGraph, a system that represents the scientific literature as a knowledge graph and uses a Graph Neural Network (GNN) to perform edge classification directly on that graph. The graph is constructed as follows:

- **Nodes** represent individual papers, each characterised by metadata-derived features such as publication year, venue, and domain. These features ground each node in its bibliometric context, allowing the model to reason about temporal and disciplinary proximity.
- **Edges** represent citation relationships and carry as their primary feature the sentence-level embedding of the citing sentence, extracted from the SciCite dataset. This embedding encodes the local rhetorical function of the citation as expressed by the author.
- **Edge labels** follow the SciCite taxonomy, background, method, and result comparison, and constitute the classification target. These three categories reflect fundamentally different epistemic roles a citation can play in scientific discourse.

By aggregating information from neighbouring nodes and edges, the GNN goes beyond the plain-text baselines of existing SciCite models: a citation’s intent is not inferred in isolation, but in the context of how the cited paper is used by the rest of the community. Publication years, required to establish temporal ordering for the second-hand citation detection task, are retrieved from the Semantic Scholar API using the paper IDs already present in SciCite.

2 Goals

1. Graph Construction.

Build a citation knowledge graph from the SciCite dataset (loaded via Hugging-Face). Each paper becomes a node enriched with bibliometric features retrieved from the Semantic Scholar API using the paper IDs already present in SciCite. Each citation becomes a directed edge carrying the embedding of the citing sentence as its feature.

2. GNN Edge Classifier.

Train a GNN edge classifier using PyTorch Geometric that aggregates neighbourhood information to predict the SciCite intent label of each citation. A text-only baseline (operating solely on the citing sentence embedding, without any graph context) is trained alongside to quantify the contribution of the graph structure.

3. Second-Hand Citation Detection.

Leverage the trained GNN for second-hand citation detection: flag edges whose classification confidence is anomalously low, or whose learned embedding is more similar to an earlier, structurally analogous edge than would be expected under independent authorship. For each flagged citation, produce a local explanation identifying the likely source edge, supported by temporal ordering derived from publication years.

4. Evaluation.

Assess the intent classifier on the SciCite test split using macro F1. Assess the second-hand citation detector on a manually labelled sample, reporting precision and recall. Where applicable, ablation results isolating the contribution of graph context are included.

3 Expected Deliverables

- **Source code** hosted on the provided GitLab repository. The main deliverable is a working software system offering two core functionalities, given a set of cited papers:

1. retrieve and rank structurally similar papers based on their position and role in the citation graph.
2. detect second-hand citations by flagging edges whose learned representation is better explained by an earlier, structurally analogous citation.

Dependency management is handled via Poetry or pip, ensuring a fully reproducible environment. The repository includes a pytest test suite and a GitLab CI configuration that runs tests and linting automatically on each push.

- **Final report** written in Markdown or $\text{\LaTeX}$, documenting the full pipeline.